

Log Anomaly Detection with Hawkes-Process Timing and LLM Triage on System Logs

A detailed, honestly-validated study on public log data

Internship project report

July 16, 2026

Abstract

This report documents an end-to-end anomaly-detection system for system logs, built on public data only (the loghub HDFS and BGL datasets). The application domain is cybersecurity operations, but the primary objective is a technically strong, rigorously validated study of *event/time-series* data. The system has three parts: (i) an unsupervised **autoencoder detector** whose reconstruction error is the anomaly score, benchmarked against an Isolation Forest baseline; (ii) a **univariate Hawkes point process** fit by maximum likelihood to model the self-excitation (clustering) of log events, validated with the time-rescaling theorem, whose conditional intensity is proposed as a detector feature; and (iii) an **LLM triage agent** that deduplicates and correlates alerts into ranked, explained incidents. Validation is deliberately strict: a never-shuffled temporal train/test split, precision–recall AUC rather than accuracy, a headline metric of *false alarms per day at a fixed detection rate*, and an explicit check for performance drift across the test period. We report what worked *and what did not*: HDFS detection is a strong reproduction of known results (PR-AUC 0.987) rather than a novelty; the Hawkes branching ratio on BGL is near-critical ($\eta \approx 0.97\text{--}1.0$), a genuine quantitative finding, but a single-exponential kernel underfits the deepest inter-event gaps; the timing feature roughly *halves* false alarms on BGL though absolute detection there is hard; and extreme-value thresholding, the principled tool, does not fit HDFS’s discrete scores, so a distribution-free alternative is used there instead. The intellectual honesty of these findings is treated as a feature of the work, not a defect.

Contents

1	Introduction: what, why, and how	1
2	Background and related work: prior art vs. our contribution	2
3	Data: two datasets, and why both are necessary	3
4	Log parsing with Drain	4
5	Exploratory analysis: the measurements that shaped the design	4
6	Detection on HDFS	5
6.1	Unit, features, and the validation protocol	5
6.2	Models: baseline vs. autoencoder	5
7	The mathematical core: Hawkes process on BGL	6
7.1	Model	6
7.2	Estimation	6
7.3	Correctness before trust	7
7.4	Validation: the time-rescaling theorem	7
7.5	Results	7

8	Does the timing signal actually help detection?	8
9	Adaptive thresholding: stabilising false alarms under drift	9
9.1	The problem	9
9.2	Extreme-Value Theory (the principled tool)	9
9.3	An honest negative finding, and the fix	10
10	LLM triage: from alerts to ranked incidents	10
11	Dashboard	11
12	Results at a glance	12
13	Honest limitations	12
14	Conclusion and future work	12

1 Introduction: what, why, and how

What. We build a pipeline that ingests raw system logs and produces a ranked list of explained security/operations incidents. Concretely: raw logs $\rightarrow$ structured events (log parsing) $\rightarrow$ features $\rightarrow$ an unsupervised detector $\rightarrow$ a self-calibrating alert threshold $\rightarrow$ an LLM triage layer $\rightarrow$ a dashboard. Alongside the detector runs the mathematical core: a Hawkes process that models the *timing* of events.

Why. System logs are the primary telemetry of any distributed system, and anomalies in them (failures, attacks, cascading faults) are rare, unlabeled, and clustered in time. Three properties make this a good vehicle for a rigorous time-series study. First, anomalies are $\sim 0.1\text{--}3\%$ of the data, so *how* you measure matters enormously—accuracy is meaningless and precision–recall under class imbalance is mandatory. Second, log events *cluster*: one failure triggers cascades of related events, which is exactly the regime a self-exciting point process (Hawkes) is designed for. Third, the data is non-stationary, so a detector that looks good on an aggregate can be useless in practice; measuring *drift* is essential. The application is cybersecurity; the transferable skill is disciplined modeling and validation of event data.

How. Everything is unsupervised at training time—labels are used *only* for evaluation—and everything is validated on a strict temporal split. We implement the Hawkes machinery ourselves (the standard `tick` library has no wheels for modern Python) and, crucially, prove it correct on simulated data before trusting it on real data. Every non-trivial design decision in this report was *driven by a measurement*, not assumed; where the data contradicted a plan, we changed the plan and documented why.

A note on scope and honesty. A recurring theme is the distinction between *what has already been done* by the community and *what this project contributes*. We are explicit about it throughout, and we summarise it in Section 2. In particular, HDFS anomaly detection with count features and an autoencoder is one of the most reproduced results in the field; our contribution is not that number but the timing model, the validation discipline, the drift-aware self-calibrating threshold, and the triage layer.

2 Background and related work: prior art vs. our contribution

This section separates the state of the art from what we did, component by component. Table 1 is the summary; the prose gives the context.

Log parsing. Turning free-text log lines into a finite vocabulary of *event templates* is a solved problem. **Drain** (He et al., 2017) is a fixed-depth parse-tree parser that is the de-facto standard, distributed in the **logparser** toolkit with benchmark configurations per dataset. *Prior art.* **What we did:** we use Drain unchanged with the published loghub configuration—parsing is infrastructure here, not a research question, and using the standard configuration keeps our templates comparable to published results.

Log anomaly detection. There is a large literature. Classical approaches build a per-session *event-count matrix* and apply PCA, Invariant Mining, or Isolation Forest (the **loglizer** toolkit, He et al., 2016). Deep approaches model event *sequences*: **DeepLog** (Du et al., 2017) and **LogAnomaly** (Meng et al., 2019) use LSTMs to predict the next event and flag surprises; autoencoders on count vectors are a common unsupervised baseline. On HDFS specifically, these methods routinely achieve F1 / PR-AUC in the 0.95–0.99 range, and there are dozens of public reproductions. *Prior art.* **What we did:** we reproduce the count-vector autoencoder (PR-AUC 0.987) precisely because it is a known-good baseline—it validates our pipeline and gives the triage layer real anomalies to consume. We do *not* claim it as a contribution. Our detection-side contributions are the *validation protocol* and the *drift analysis* that most reproductions omit (they typically shuffle and report accuracy/F1).

Point processes for logs. Hawkes processes (Hawkes, 1971) are standard in seismology, finance (order-flow, market microstructure), and social media, and the time-rescaling theorem (Brown et al., 2002) is the standard goodness-of-fit tool for point processes. Their use as a *feature source for log anomaly detection* is uncommon—most log-anomaly work discards fine-grained timing and models only which events occur. *Partial prior art.* **What we did:** we fit a univariate exponential-kernel Hawkes process to BGL event times, estimate the branching ratio, validate with time-rescaling, and *test* whether the resulting self-excitation signal improves detection. This timing angle—and the honest measurement of whether it helps—is the intellectual centerpiece.

Adaptive thresholding. Extreme-Value Theory (EVT) with Peaks-Over-Threshold and the Generalized Pareto Distribution is classical (Pickands, 1975; Balkema–de Haan). Applying it to *streaming* anomaly scores is the **SPOT/DSPOT** method (Siffer et al., 2017). *Prior art.* **What we did:** we implement POT/GPD and a streaming trailing-window variant, and we report a specific, honest negative finding—EVT assumes a continuous tail, but the HDFS autoencoder scores are strongly discrete, so GPD extrapolation misbehaves there; we therefore use a distribution-free rolling quantile on HDFS and reserve EVT for BGL’s continuous scores. The drift-stabilisation *demonstration* on measured drift is ours.

LLM triage. Using LLMs to summarise or triage alerts is an active, recent, and largely engineering-driven area. *Emerging prior art.* **What we did:** we build a triage agent whose novel element is that *correlation into incidents is grounded in the self-excitation model*—a cascade of temporally clustered anomalies becomes a single incident—rather than being an ad-hoc time-window heuristic, and we emit strictly-typed JSON via structured outputs.

Table 1: Prior art vs. our contribution, by component.

Component	Already established (prior art)	What we did / contributed
Parsing	Drain is the standard parser (loghub).	Use it unchanged; standard config for comparability.
Detection	Count-vector + autoencoder / Iso-Forest on HDFS is heavily reproduced (0.95–0.99).	Reproduce as a validated baseline; contribution is the strict protocol + drift analysis around it.
Timing	Hawkes + time-rescaling standard in other fields; rare for logs.	Fit on BGL, estimate branching ratio, validate, and <i>measure</i> whether timing aids detection.
Thresholding	EVT/POT and SPOT/DSPOT for streams.	Implement both; report that EVT fails on discrete HDFS scores; demonstrate drift stabilisation.
Triage	LLMs for alert summarisation (recent).	Incident correlation grounded in self-excitation; typed JSON output.
Validation	Temporal splits and PR-AUC known in principle, often skipped in practice.	Applied rigorously end-to-end; FA/day headline; per-period drift.

3 Data: two datasets, and why both are necessary

We use *two* loghub datasets because they play complementary roles that neither can fill alone. This was decided by measurement, not preference.

Table 2: The two datasets and their roles.

	HDFS	BGL
Role	Detection + triage showcase	Hawkes-process centerpiece
Raw size	11,175,629 lines	4,713,493 lines
Templates (Drain)	48	429
Time span	~1.6 days	~214.7 days
Labels	per <i>block</i> (2.93% anomalous)	per <i>line</i> (7.39% alerts)
Timestamp resolution	1 second	microsecond

The timestamp-resolution problem (decisive). A continuous-time Hawkes process requires a strict ordering of event times. We measured whether the data provides one:

- **HDFS** carries only 1-second timestamps at ~80 events/second. As a result, **98.8%** of consecutive events in the global stream share a timestamp; *within* a single block, 90.5% still tie, and every block’s entire life is ≤ 54 s (median 7 s), occupying ~2 distinct seconds. There is essentially no temporal ordering to fit, at any level. Adding sub-second jitter would fabricate the very structure the model is meant to measure (the branching-ratio estimate would become an artifact of the jitter), so we rejected it.
- **BGL** carries genuine *microsecond* timestamps (e.g. 2005-06-03-15.42.50.675872), with 0% ties and events naturally spread over seconds—exactly what a continuous-time Hawkes fit needs.

Consequently HDFS hosts detection (its clean per-block labels are ideal for false-alarms-per-day) and BGL hosts the Hawkes work (its timing is the only place the math is well-posed). Using two independent benchmarks is a strength, not redundancy.

4 Log parsing with Drain

Raw lines are unstructured text; the rest of the pipeline needs structure. Drain mines a fixed vocabulary of templates:

```
081109 203515 148 INFO ... PacketResponder 1 for block blk_388...terminating
→ template "PacketResponder <*> for block <*> terminating"
```

Each line becomes (timestamp, event_id, template, ...). HDFS lines additionally carry a block_id (the session key for its per-block labels); BGL lines carry an inline alert label and the microsecond timestamp. The full HDFS log parses to 48 templates and BGL to 429. A coherence check confirms correctness: the HDFS allocateBlock template occurs exactly 575,061 times—once per block—matching the label count exactly. (Code: `src/logtrriage/parsing/.`)

5 Exploratory analysis: the measurements that shaped the design

Three findings changed the plan.

(1) **Events cluster hard—self-excitation is real.** The Fano factor (variance/mean of per-minute counts; = 1 for a Poisson process) is 5.3 on the HDFS sample and 698–2286 on BGL. This is the empirical justification for a Hawkes model *before any fit*. Figure 1 (left) shows the HDFS event timeline: long near-zero baselines punctuated by sharp bursts—the visible signature of self-excitation.

(2) **The HDFS timestamp problem** (Section 3)—which moved the Hawkes work to BGL.

(3) **Anomaly-rate drift.** On the temporal split (Section 6), the earlier 70% of blocks are 3.26% anomalous and the later 30% are 2.16%—a base rate that is *not* stationary, foreshadowing the drift results. Figure 1 (right) shows the HDFS block-session size distribution (median 19 events/block), whose long tail is disproportionately the anomalous blocks.

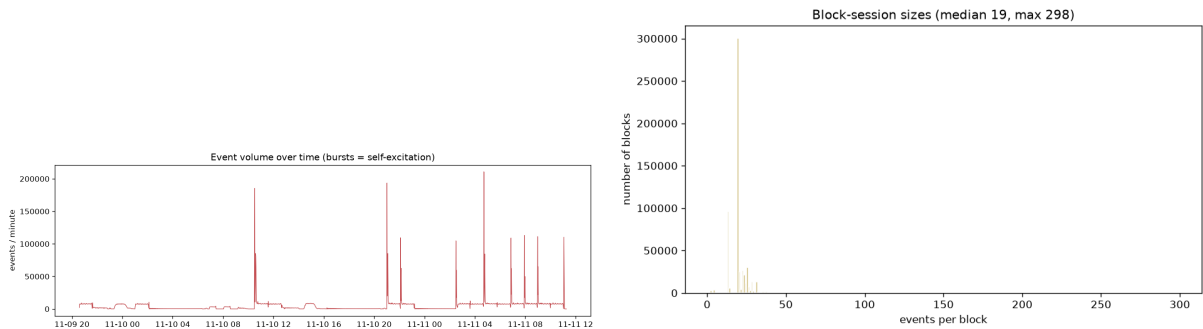


Figure 1: Left: HDFS event volume over time—flat baseline with sharp bursts (self-excitation). Right: block-session sizes (median 19; the tail is disproportionately anomalous).

6 Detection on HDFS

6.1 Unit, features, and the validation protocol

Because HDFS labels are per *block* and a block lives ≤ 54 s, the detection unit is the **block session**: all of a block’s events, grouped, represented as a **48-dimensional vector of template counts** (the standard loglizer “event count matrix”). There are 575,061 blocks.

The validation protocol is the heart of the project:

- **Strict temporal split, never shuffled**: blocks ordered by start time; earliest 70% train, latest 30% test. No test information leaks into training, scaling, or thresholds. The split boundary is 2008-11-11 06:00:46; train = 402,543 blocks (3.258% anomalous), test = 172,518 (2.158%).
- **PR-AUC, not accuracy or ROC**: positives are $\sim 2\%$, so accuracy $\approx 98\%$ is uninformative and ROC is optimistic under imbalance.
- **Headline metric: false alarms per day at a fixed detection rate**, computed on the *actual* elapsed test time (blocks are not uniform in time).
- **Drift**: every metric is also reported over consecutive chronological slices of the test period.

6.2 Models: baseline vs. autoencoder

Both are unsupervised and scored through the *same* metrics harness.

- **Isolation Forest** (baseline): fit on $\log(1 + \text{counts})$; the anomaly score is $-\text{score_samples}$.
- **Autoencoder** (detector): a multilayer perceptron $48 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow 48$ with ReLU, trained *only on normal train blocks* (labels never enter training); per-block reconstruction MSE is the anomaly score. Preprocessing is fit on normal train data only, so no test information leaks.

Table 3: HDFS detection, test split (172,518 blocks, 2.16% anomalous).

Metric	Isolation Forest	Autoencoder
PR-AUC (average precision)	0.748	0.987
Precision @ 90% recall	0.556	0.961
False alarms / day @ 90% recall	$\sim 12,700$	~ 650

The autoencoder cuts false alarms $\sim 20\times$ at the same detection rate, and its discrimination is *stable under drift* (PR-AUC 0.99+ in every test sub-period, where the baseline swung 0.69–0.98). Figure 2 shows the precision–recall curves.

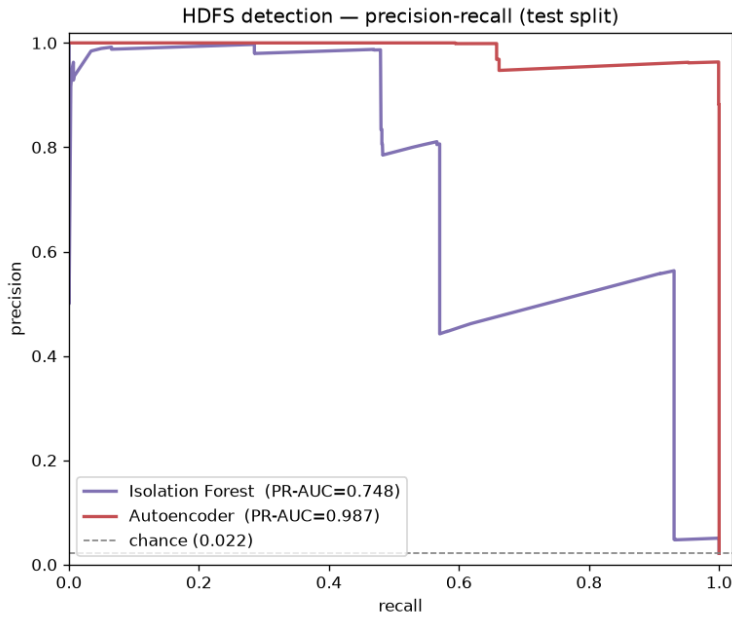


Figure 2: HDFS precision–recall, test split. The autoencoder (PR-AUC 0.987) dominates the Isolation Forest baseline (0.748), holding near-perfect precision until ~ 0.95 recall.

Honest framing. This is a strong *reproduction*, not a novelty (Section 2). Its role is to validate the pipeline and to be the substrate for the un-commodity parts: the drift analysis, the adaptive threshold, and triage.

7 The mathematical core: Hawkes process on BGL

7.1 Model

A univariate Hawkes process with exponential kernel has conditional intensity

$$\lambda(t) = \mu + \sum_{t_i < t} \alpha e^{-\beta(t-t_i)}, \quad \mu, \alpha, \beta > 0, \quad (1)$$

so each event raises the instantaneous rate by α , decaying at rate β . The **branching ratio**

$$\eta = \int_0^\infty \alpha e^{-\beta s} ds = \frac{\alpha}{\beta} \quad (2)$$

is the expected number of *direct* offspring per event; the process is stationary/subcritical iff $\eta < 1$. This one number summarises how cascade-prone the stream is.

7.2 Estimation

On an observation window $[0, T]$ with events $\{t_i\}$, the log-likelihood is

$$\ell(\mu, \alpha, \beta) = \sum_i \log \lambda(t_i) - \int_0^T \lambda(s) ds, \quad (3)$$

where the integral is closed-form, $\int_0^T \lambda = \mu T + \frac{\alpha}{\beta} \sum_i (1 - e^{-\beta(T-t_i)})$, and the sum term is computed in $O(n)$ via Ogata’s recursion $A_i = e^{-\beta(t_i - t_{i-1})}(1 + A_{i-1})$, $\lambda(t_i) = \mu + \alpha A_i$. We maximise ℓ in log-space (so positivity is automatic) with L-BFGS-B.

7.3 Correctness before trust

Because this is the centerpiece, it is *proven on simulated data before touching BGL* (`tests/test_hawkes.py`): we simulate a process with known (μ, α, β) via Ogata thinning, fit it, and confirm parameter recovery; and we confirm the goodness-of-fit test *passes* a correct fit and *rejects* a deliberately wrong model. We implemented all of this directly with `scipy` because the `tick` library has no wheels for modern Python; the mathematics is library-independent.

7.4 Validation: the time-rescaling theorem

If the fitted model is correct, the compensator $\Lambda(t) = \int_0^t \lambda(s) ds$ transforms the events into a unit-rate Poisson process, so the rescaled inter-event gaps $\Delta\tau_i = \Lambda(t_i) - \Lambda(t_{i-1})$ are i.i.d. $\text{Exp}(1)$. We test this with a Kolmogorov–Smirnov statistic and, more informatively, a QQ plot against $\text{Exp}(1)$. A branching ratio is meaningless if the fit fails rescaling—this is the model-independent check the project’s rigor rests on.

7.5 Results

Table 4: Hawkes fits on BGL windows.

BGL window	Events	$\eta = \alpha/\beta$	KS stat	mean rescaled gap
2005-09-03 (moderate)	6,858	0.971	0.095	1.0001
2005-06-11 (fault cascade)	152,669	0.9998	0.431	1.0000

Reading. The **branching ratio is near-critical everywhere** ($\eta \approx 0.97\text{--}1.0$): BGL’s event stream is strongly, almost self-sustaining, self-exciting—a precise *estimated* cascade strength, not just “bursty.” The QQ plots (Figure 3, right) show the body of the rescaled-gap distribution on the Exp(1) diagonal but the *upper tail* deviating upward: the deepest quiet gaps are longer than a single exponential predicts. Interpretation: right after a burst the fitted excitation predicts many events; when silence follows, the integrated intensity over that gap is large, producing outlying rescaled gaps. The effect is mild on the calm window (KS 0.095) and severe on the extreme cascade (KS 0.431, η pinned at ~ 1). A single exponential kernel cannot hold both tight bursts and deep lulls. The mean rescaled gap of exactly 1.0000 confirms the compensator is correctly normalised even where the shape deviates; at 10^5 points the KS test rejects almost any parametric model, so we lean on the QQ shape and the magnitude, not the p -value.

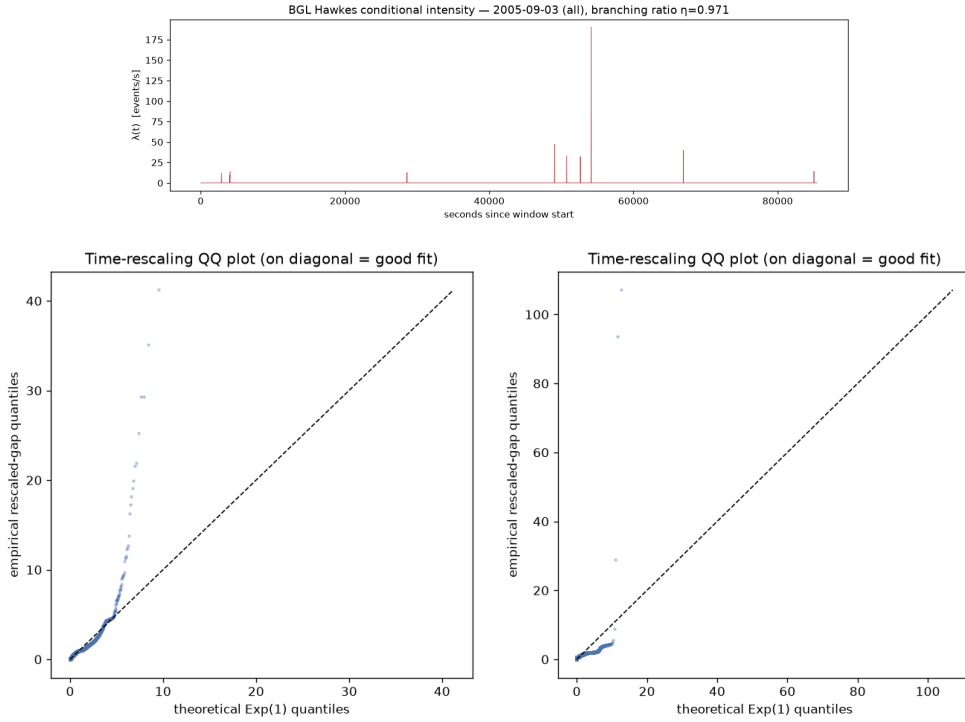


Figure 3: Top: fitted conditional intensity $\lambda(t)$ on 2005-09-03—near-zero baseline with sharp spikes (self-excitation). Bottom: time-rescaling QQ plots for the moderate window (left, KS 0.095) and the extreme cascade (right, KS 0.431); both show diagonal body, deviating upper tail.

This is precisely the kind of result a rigorous study should produce: a validated estimate *plus* an honest characterisation of where the model breaks, which directly motivates a sum-of-exponentials kernel as future work.

8 Does the timing signal actually help detection?

This is the question the BGL half exists to answer. On fixed 60-second BGL windows we detect alert-containing windows three ways with the *same* model (Isolation Forest) and the *same* temporal split, changing only the features:

- **content**: counts of the top-50 event templates (bag of events, no timing);
- **timing**: the per-window Hawkes self-excitation $A(t) = \sum_{t_i < t} e^{-\beta(t-t_i)}$ at two decay scales (1 s, 30 s) plus the event count—computed from event *times only*, hence leakage-free;
- **content + timing**: both.

Table 5: BGL window detection, feature ablation (test split, 4.71% anomalous).

Feature set	PR-AUC	Precision @ 90% recall	False alarms / day
content	0.081	0.046	114
timing	0.087	0.083	61
content+timing	0.090	0.071	72
chance	0.047	—	—

The honest, nuanced answer. Absolute detection is *weak* here (~ 0.09 PR-AUC, $\sim 2\times$ chance): detecting BGL fault windows at this resolution is genuinely hard, and we do not oversell it—at 60 s most alert windows fold a few alert lines into heavy normal traffic, so the unsupervised signal is faint. *But* the timing signal is the more useful one and it **cuts false alarms**: at 90% recall, timing roughly *halves* the false-alarm rate versus content ($114 \rightarrow 61$ per day) and nearly doubles precision. Figure 4 shows content collapsing to the chance line at high recall while timing holds $\sim 2\times$ chance across the operational range. So the Hawkes self-excitation, as a feature, is the single most useful signal and moves the metric the project cares about—it just does not, alone, turn window-level BGL detection into a strong detector. A rigorously-validated “timing beats content and halves false alarms, on a hard task” is a real finding; a manufactured 0.98 would not be.

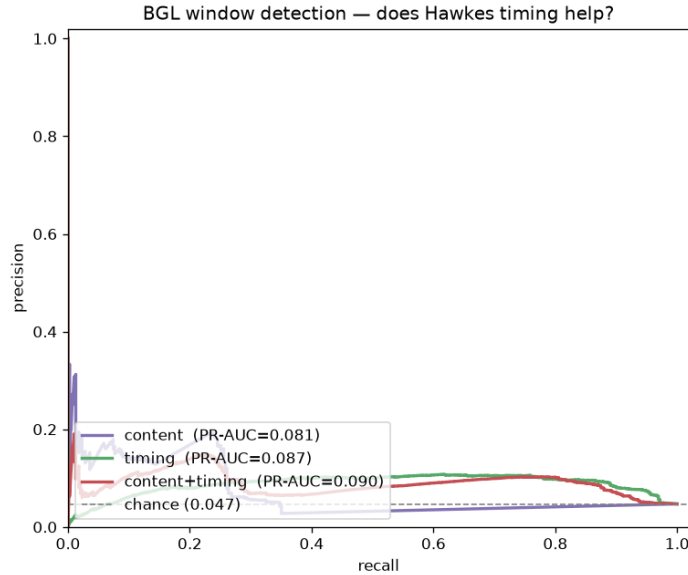


Figure 4: BGL window detection PR curves. All are weak in absolute terms, but timing (green) and content+timing (red) dominate content (purple), which collapses to chance at higher recall.

9 Adaptive thresholding: stabilising false alarms under drift

9.1 The problem

A detector outputs scores; a threshold turns them into alerts. The drift analysis exposed the real operational weakness: at a *fixed* threshold, the false-alarm rate wanders as the score distribution drifts, even while discrimination stays high. On the HDFS autoencoder scores, across a single 5-hour test window, a fixed threshold’s false alarms swing from 3,664 to **861,686** per day—flagging up to 100% of blocks mid-period (Figure 5).

9.2 Extreme-Value Theory (the principled tool)

By the Pickands–Balkema–de Haan theorem, exceedances of a high threshold t converge to a Generalized Pareto Distribution (GPD):

$$\Pr(X - t > x \mid X > t) \approx \left(1 + \frac{\gamma x}{\sigma}\right)^{-1/\gamma}. \quad (4)$$

Fitting (γ, σ) to observed exceedances and inverting for a target tail probability q gives the self-calibrating alert level

$$z_q = t + \frac{\sigma}{\gamma} \left[\left(\frac{qn}{N_t} \right)^{-\gamma} - 1 \right], \quad (5)$$

with n the sample size and N_t the number of exceedances. Holding q fixed while re-estimating (t, γ, σ) from recent data keeps the flagged rate—hence the false-alarm rate—steady as the distribution moves. We implement this (**AdaptivePot**, GPD fit by method of moments for $O(1)$ streaming updates) and verify on simulated data that it controls the exceedance rate and tracks drift.

9.3 An honest negative finding, and the fix

Applied to HDFS, GPD extrapolation misbehaves (thresholds fall below the data floor and flag everything). The cause is real: the HDFS autoencoder scores are strongly *discrete*—58.7% of normal blocks reconstruct to the *identical* value, because normal HDFS blocks are so repetitive—which violates GPD’s continuous-tail assumption. We therefore use the distribution-free **rolling quantile** on HDFS, and reserve EVT for BGL’s *continuous* timing scores, where its extrapolation to small q genuinely earns its place.

Table 6: HDFS false-alarm stability under drift (autoencoder scores, target flag rate $q = 3\%$).

	Fixed threshold	Adaptive (rolling quantile)
False alarms / day (range over test)	3,664– 861,686	0–7,584
Flagged fraction	up to 100%	stable $\approx 2\%$
Recall	1.00	0.72

The adaptive threshold holds the alert rate steady at the transparent cost of recall in periods where anomalies surge past the flag budget—the honest operating-point tradeoff, now explicit and tunable via q .

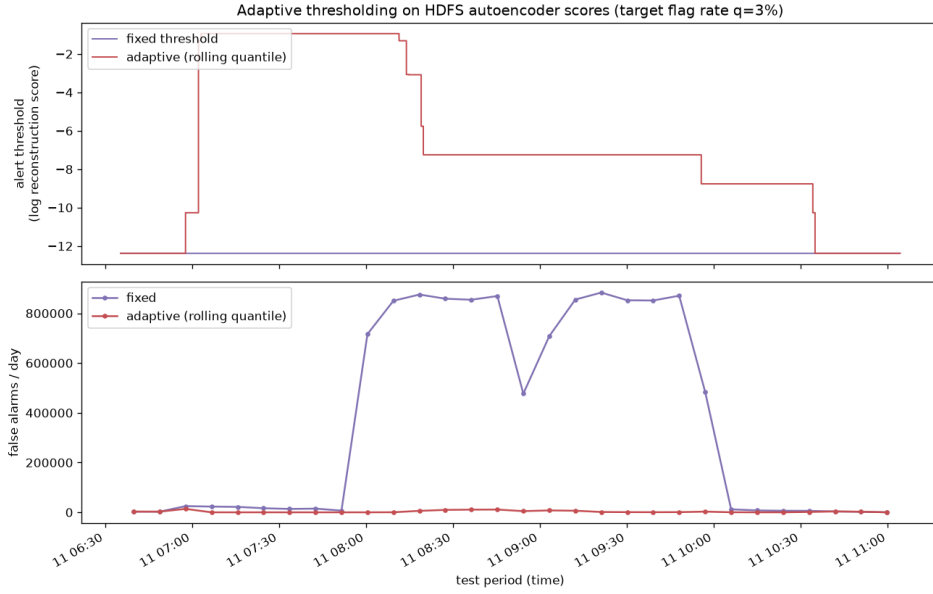


Figure 5: Top: the fixed threshold stays put while the normal-block scores drift up mid-period; the adaptive threshold rises to track them. Bottom: the fixed threshold’s false alarms explode to $\sim 860\text{k/day}$ mid-period; the adaptive threshold holds the alert rate near the target.

10 LLM triage: from alerts to ranked incidents

Paging an analyst once per flagged block is unusable. The triage agent reduces volume, then explains, in three steps:

1. **Deduplicate**: flags with the same signature (entity + template multiset) collapse into one, with a count.
2. **Correlate into incidents**: flags close in time are grouped into a single incident. This is the Hawkes idea made operational—self-exciting events are causally linked, so a cascade becomes *one* incident, not 200 pages. This grounding in the timing model is what distinguishes it from an ad-hoc window heuristic.
3. **Explain and rank**: each incident is classified into strictly-typed JSON (**priority**, **category**, **summary**, **recommended_action**, **explanation**) and ranked P1→P4. The explainer is plug-gable: a deterministic rule-based backend (category from template keywords, priority from severity + cascade size) runs self-contained, and an **Anthropic backend** (claude-opus-4-8, JSON-schema structured output) drops in when a credential is configured, falling back automatically. There is *one* model call per incident, not per raw flag.

On the top-200 flagged HDFS blocks: **200 raw flags** $\rightarrow$ **90 incidents**, prioritised P1(13)/P2(53)/P3(2)/P4(2) across data-loss / replication / network categories (Figure 6).

Note. This environment had no model credential, so the demo ran on the rule-based explainer; the Anthropic path is written to the current API and activates with a key.

11 Dashboard

A Streamlit application (`app/dashboard.py`) surfaces the whole pipeline in four tabs—Detection (metrics + PR curve), Thresholding (the drift-stabilisation plot), Hawkes (intensity + QQ), and Incidents (the ranked table with per-incident drill-down). It was verified to start and render headless (Figure 6).

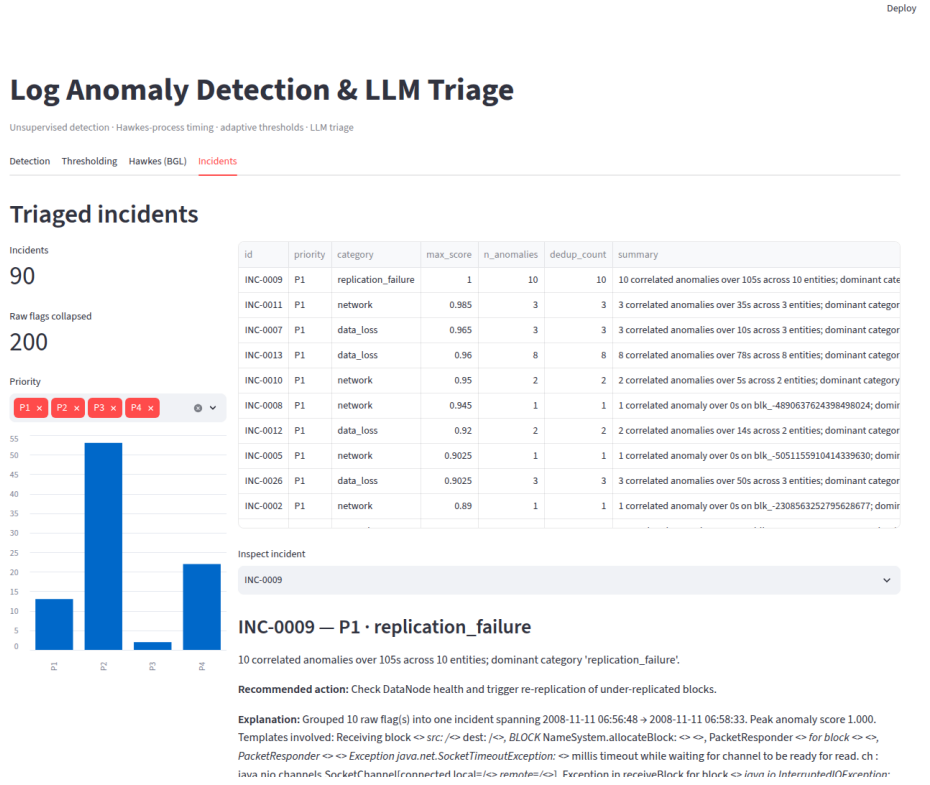


Figure 6: The dashboard’s Incidents tab: 200 raw flags collapsed to 90 ranked incidents, with priority distribution and per-incident drill-down (summary, recommended action, explanation).

12 Results at a glance

Table 7: Consolidated results.

Result	Value
HDFS parsed	11.2M lines → 48 templates
BGL parsed	4.71M lines → 429 templates
Autoencoder PR-AUC (test)	0.987 (baseline 0.748)
False alarms/day @ 90% recall	~650 (baseline ~12,700)
Hawkes branching ratio (BGL)	$\eta \approx 0.97\text{--}1.0$ (near-critical)
Timing feature effect (BGL)	halves false alarms vs. content
Fixed → adaptive threshold	FA/day swing $\times 235$ → held near target
Triage reduction	200 flags → 90 ranked incidents
Unit tests	34 passing

13 Honest limitations

- **HDFS detection is a reproduction**, not a contribution; its role is validation and substrate, and we say so plainly.
- **BGL window detection is weak in absolute terms** (~ 0.09 PR-AUC); the timing result is a *relative* win (fewer false alarms) on a hard task.
- **The single-exponential Hawkes kernel underfits** BGL’s deepest lulls (time-rescaling upper tail); a sum-of-exponentials kernel is the fix.
- **EVT does not fit the discrete HDFS scores**; the rolling quantile is used there, and EVT is reserved for continuous scores.

- **The LLM triage backend was not exercised live** (no credential in this environment); only the rule-based path ran end-to-end here.
- Window size, feature vocabulary, and decay scales are first choices, not swept.

14 Conclusion and future work

We built a complete, unsupervised log-anomaly pipeline with a stochastic-process core and an LLM triage layer, validated with a discipline (temporal split, PR-AUC, false-alarms-per-day, drift) that most public reproductions omit. The mathematical centerpiece—a Hawkes process fit and validated on BGL—yields a concrete, interpretable finding (near-critical branching ratio) and an honest account of where a single-exponential kernel breaks. The timing signal it produces measurably reduces false alarms, even though absolute detection on BGL is hard. Throughout, we separated *what the community has already established* from *what we contributed*, and we reported negative and nuanced results as first-class outcomes.

Future work. (1) A richer Hawkes kernel (sum-of-exponentials or power-law) to fix the upper-tail misfit. (2) Use the time-rescaling *residual* directly as a calibrated timing-anomaly detector, with EVT thresholding on those continuous scores. (3) Sweep window size and decay scales; try the autoencoder in place of Isolation Forest for the timing ablation. (4) Exercise the LLM triage end-to-end with a real model and evaluate incident quality against held-out labels.

References

- [1] P. He, J. Zhu, Z. Zheng, M. R. Lyu. “Drain: An Online Log Parsing Approach with Fixed Depth Tree.” *ICWS*, 2017.
- [2] S. He, J. Zhu, P. He, M. R. Lyu. “Experience Report: System Log Analysis for Anomaly Detection.” *ISSRE*, 2016.
- [3] M. Du, F. Li, G. Zheng, V. Srikumar. “DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning.” *ACM CCS*, 2017.
- [4] W. Meng et al. “LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs.” *IJCAI*, 2019.
- [5] A. G. Hawkes. “Spectra of Some Self-Exciting and Mutually Exciting Point Processes.” *Biometrika*, 1971.
- [6] Y. Ogata. “On Lewis’ Simulation Method for Point Processes.” *IEEE Trans. Inf. Theory*, 1981.
- [7] E. N. Brown, R. Barbieri, V. Ventura, R. E. Kass, L. M. Frank. “The Time-Rescaling Theorem and Its Application to Neural Spike Train Data Analysis.” *Neural Computation*, 2002.
- [8] A. Siffer, P.-A. Fouque, A. Termier, C. Largouët. “Anomaly Detection in Streams with Extreme Value Theory.” *KDD*, 2017.
- [9] S. He, J. Zhu, P. He, M. R. Lyu. “Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics.” *ISSRE*, 2023.